

Coding Standards

Document: docs/architecture/CODING_STANDARDS.md

System: HackNest — Student Operating System

Status: Foundational / Authoritative — closes a previously-missing Governance-layer document

Type: Standards Specification Only — No Implementation Code

Follows README.md and MASTER_ARCHITECTURE.md exactly. Per MASTER_ARCHITECTURE.md Section 9, this document is the sole authoritative owner of code-level naming, structure, and tooling conventions — the one concern MASTER_ARCHITECTURE.md Section 8 explicitly deferred rather than encroaching on. This document contains conventions only; it contains no actual source code.

1. Purpose

MASTER_ARCHITECTURE.md governs domain, document, and role naming at the architecture level. This document extends that discipline down to the code that implements it, so that a domain named "Teams" in DATABASE_ARCHITECTURE.md is recognizable as "Teams" everywhere it appears in code, not renamed inconsistently per file.

2. Language and Stack Conventions

- **TypeScript** is used throughout, in strict mode; no file relies on implicit `any`.
 - **Next.js 15** conventions govern file-based routing, Server Components, Server Actions, and Route Handlers, consistent with the Server-first principle in MASTER_ARCHITECTURE.md Section 2.
 - **Supabase client usage** follows the Server vs. Client responsibility split owned by DATABASE_ARCHITECTURE.md Section 18 — this document does not redefine that boundary, only the file/module conventions for observing it.
-

3. Naming Conventions

Element	Convention	Example
Domain-owned folders	lowercase, plural, matching DATABASE_ARCHITECTURE.md Section 4 exactly	teams/ , projects/ , hackathons/
React components	PascalCase	TeamCard.tsx
Server Actions	camelCase verb-first function names	createTeam , updateProjectStatus
Route Handlers	lowercase route segments matching the domain name	/api/v1/teams
Types/Interfaces	PascalCase, suffixed by role where useful	Team , TeamMember , CreateTeamInput
Database-adjacent field names (as referenced in code)	camelCase in TypeScript, matching the conceptual field names in DATABASE_ARCHITECTURE.md Section 9 (e.g., createdAt , deletedAt)	createdAt
Environment variables	SCREAMING_SNAKE_CASE, prefixed by concern	SUPABASE_SERVICE_ROLE_KEY
Test files	co-located, suffixed .test.ts / .test.tsx	TeamCard.test.tsx

4. Folder Organization (Application Level)

Extending MASTER_ARCHITECTURE.md Section 8's documentation-only folder structure to the application itself:

```

app/
  (domain-name)/      # one folder per domain, matching
DATABASE_ARCHITECTURE.md Section 4
  actions.ts          # Server Actions for this domain
  components/         # domain-scoped UI components
  page.tsx            # domain's primary route, where
applicable

```

```

lib/
  domains/
    (domain-name)/
      types.ts           # domain type definitions
      queries.ts         # data-access functions (server-only)
      validation.ts      # Zod schemas for this domain
      shared/           # cross-domain utilities only – never
domain-specific logic
components/
  ui/                  # design-system primitives per
UI_DESIGN_SYSTEM.md
tests/
  (mirrors app/ and lib/ structure)

```

A file's location must match its domain ownership per

`DATABASE_ARCHITECTURE.md` Section 4 – no domain's logic lives inside another domain's folder, consistent with the Module Boundaries in

`MASTER_ARCHITECTURE.md` Section 4.

5. Code Structure Rules

- **One domain, one owner folder.** Cross-domain composition (e.g., Feed reading from Teams/Projects/Hackathons) lives in the composing domain's own folder (`feed/`), calling into other domains' exposed query functions – never reaching directly into another domain's internal files.
- **No direct Supabase client calls from client components.** All data access is server-side, per the Server-first principle – client components call Server Actions or fetch from Route Handlers, never instantiate a Supabase client themselves for authenticated data.
- **Validation schemas live beside the domain they validate**, not centralized in one giant file, so a domain's contract (per `API_ARCHITECTURE.md`) stays reviewable in one place.
- **No magic numbers for the values defined in** `MASTER_GOVERNANCE_AMENDMENT_001.md` Section 4 (rate limits, retention windows, etc.) – these are referenced from a single shared constants module, so a future revision to that governance document requires updating one place, not hunting through the codebase.

6. Linting, Formatting, and Type Checking

- A consistent linter and formatter configuration is applied repository-wide and enforced in CI per `DEPLOYMENT_ARCHITECTURE.md`'s Build Pipeline — no file is exempted.
 - TypeScript strict mode is non-negotiable; a change that requires disabling strictness to pass is treated as a design problem, not a lint exception to suppress.
 - Import order and unused-import rules are enforced automatically rather than left to reviewer discretion.
-

7. Commit and Review Conventions

- Commits reference the roadmap phase they belong to (e.g., `[Phase 007] Add team invite flow`), so implementation work stays traceable to `IMPLEMENTATION_ROADMAP.md`.
 - Any change touching a domain's data shape, API contract, or security posture requires the corresponding sign-off defined in `MASTER_GOVERNANCE_AMENDMENT_001.md` Section 5 before merge.
-

Definition of Done

- [x] Standards specification only — no implementation code.
 - [x] Follows `README.md` and `MASTER_ARCHITECTURE.md` exactly, deferring to it for domain/role naming rather than redefining it.
-

Validation Report

Internal consistency: naming, folder, and structure rules cross-reference each other without contradiction (e.g., Section 4's folder names directly reuse Section 3's naming convention). **Consistency with governing documents:** domain folder names map exactly to `DATABASE_ARCHITECTURE.md` Section 4; module boundary rules reinforce, without restating, `MASTER_ARCHITECTURE.md` Section 4; the shared-constants rule in Section 5 directly operationalizes

`MASTER_GOVERNANCE_AMENDMENT_001.md` Section 4's numeric targets.

Outstanding follow-up: `API_ARCHITECTURE.md`'s versioning-path convention (`v1`) should be cross-checked against Section 4's route-handler naming on that document's reconciliation pass (addressed next in this session).

Status: Validated – Governance Layer Complete